

MYSQL

```
CREATE DATABASE file_db;

show databases

use file_db;

-- DROP database file_db;

CREATE TABLE students(
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  student_name VARCHAR(100),
  course VARCHAR(50)
);

SHOW TABLES;
describe students;

RENAME TABLE student TO students;

ALTER TABLE students ADD email VARCHAR(100);

ALTER TABLE students MODIFY student_name VARCHAR(150);

ALTER TABLE students RENAME COLUMN student_name TO username;

ALTER TABLE students DROP COLUMN email;

TRUNCATE TABLE students;

DROP TABLE students;

-- ddl -> data definition language

-- DML -> data manipulation language (insert, update, delete)

INSERT INTO students VALUES(1, "fiit", "Java") , (2,"academy","Mern");

UPDATE students SET course = "Java" WHERE student_id = 1;

DELETE FROM students WHERE student_id = 2;

-- DQL => Data Query Language
-- SELECT

SELECT * FROM students;
```

```
SELECT * FROM students WHERE course = "Java";

-- TCL => Transaction control language

BEGIN;  -- stores the current state

UPDATE students SET course = "Java" WHERE student_id = 2;

ROLLBACK;

-- COMMIT -> stores permanently

-- DCL => Data control language

-- GRANT, REVOKE

CREATE USER 'fiit'@'localhost' IDENTIFIED BY 'password123';

SELECT user, host FROM mysql.user;

GRANT SELECT, INSERT, UPDATE ON students TO 'fiit'@'localhost';

GRANT ALL PRIVILEGES ON file_db.* TO 'fiit'@'localhost';

REVOKE SELECT, INSERT, UPDATE ON file_db.* FROM 'fiit'@'localhost';

REVOKE ALL PRIVILEGES ON file_db.* FROM 'fiit'@'localhost';

SHOW GRANTS FOR 'fiit'@'localhost';

CREATE TABLE departments(
    department_name VARCHAR(50) PRIMARY KEY,
    location VARCHAR(50)
);

INSERT INTO departments VALUES("Engineering", "New york"),
("Sales" , "New york") , ("HR", "India") , ("Marketing" , "London");

SELECT * FROM departments;
```

```

CREATE TABLE employees(
  id INT PRIMARY KEY,
  name VARCHAR(50),
  department VARCHAR(50),
  salary DECIMAL(10, 2),
  hire_date DATE
);

INSERT INTO employees VALUES(1 , "Harini" , "Engineering" , 90000.00 , '2021-01-05'),
(2 , "Imaya" , "Engineering" , 100000.00 , '2020-03-22'),(3 , "Alice" ,
"Sales" , 60000.00 , '2020-05-20');

INSERT INTO employees VALUES(4 , "Bob" , "Marketing" , 90000.00 , '2021-01-05'),
(5 , "Ganesh" , "HR" , 100000.00 , '2020-03-22'),(6 , "Giri" , "HR" , 60000.00 , '2020-05-20');

SELECT * FROM employees;

SELECT COUNT(*) AS TOTAL_EMPLOYEE FROM employees;

SELECT COUNT(*) FROM employees WHERE department = "Engineering";

SELECT COUNT(DISTINCT department) FROM employees;

SELECT SUM(salary) FROM employees;

SELECT AVG(salary) FROM employees;

SELECT Max(salary) , MIN(salary) FROM employees;

SELECT MAX(hire_date) FROM employees;

-- order by

SELECT name, department FROM employees ORDER BY name DESC;
-- GROUPBY

SELECT department , COUNT(*) as emp_count FROM employees
GROUP BY department;

-- find the average salary per department, sort by highest average

SELECT department , AVG(salary) as avg_sal FROM employees
GROUP BY department

```

```

ORDER BY avg_sal DESC;

-- find department that have more than 2 employee

SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING emp_count > 1;

-- subquery

SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- in

CREATE TABLE projects(
    project_id INT PRIMARY KEY,
    project_name VARCHAR(100) NOT NULL,
    client_email VARCHAR(100) UNIQUE
);

CREATE TABLE contractors(
    contractors_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    assigned_project_id INT,
    FOREIGN KEY (assigned_project_id) references projects(project_id)
);

INSERT INTO projects VALUES(101 , "Website Design" , 'web@client.com'),
(102, "Mobile app", "app@client.com"),
(103, "Database", 'db@client.com');

INSERT INTO contractors VALUES(1, "Alice" , 101),
(2, "Harini", 101),
(3, "Imaya", 102),
(4, "Arun", 103),
(5, "Raj" , NULL);

--INSERT INTO contractors VALUES(6, "Alice" , 999);
-- referential integrity

SELECT * FROM projects;
SELECT * FROM contractors;

```

```

-- joins -> inner join, left join, right join

SELECT contractors.name, projects.project_name
FROM contractors
INNER JOIN projects ON contractors.assigned_project_id = projects.project_id;

SELECT contractors.name, projects.project_name
FROM contractors
LEFT JOIN projects ON contractors.assigned_project_id = projects.project_id;

SELECT contractors.name, projects.project_name
FROM contractors
RIGHT JOIN projects ON contractors.assigned_project_id = projects.project_id;

-- special operators
-- find all employees whose name starts with the letter "A"

SELECT name, department
FROM employees
WHERE name LIKE 'A%';

-- second letter has a

SELECT name , department
FROM employees
WHERE name LIKE '_a%';

-- last a

SELECT name , department
FROM employees
WHERE name LIKE '%a';

-- IN

SELECT name, department
FROM employees
WHERE department IN ("HR", "Sales");

-- between
SELECT name , salary
FROM employees
WHERE salary BETWEEN 80000 AND 100000;

-- IS NULL

SELECT name

```

```

FROM contractors
WHERE assigned_project_id IS NULL;

SELECT name
FROM contractors
WHERE assigned_project_id IS NOT NULL;

-- find any employee who makes more money than the lowest
-- paid person in the engineering department

-- any all
SELECT name, salary
FROM employees
WHERE salary > ALL ( SELECT salary FROM employees WHERE department =
"Engineering");

SELECT name FROM employees
UNION ALL
SELECT name FROM contractors;

```

Theory

- **DDL (Data Definition Language)**
 - Defines database structure.
 - Commands: CREATE, ALTER, DROP, TRUNCATE, RENAME.
- **DML (Data Manipulation Language)**
 - Modifies data inside tables.
 - Commands: INSERT, UPDATE, DELETE.
- **DQL (Data Query Language)**
 - Queries data.
 - Command: SELECT.
- **TCL (Transaction Control Language)**
 - Manages transactions.
 - Commands: BEGIN, COMMIT, ROLLBACK.
- **DCL (Data Control Language)**
 - Manages permissions.
 - Commands: GRANT, REVOKE.

- **Joins**
 - INNER JOIN → returns matching rows.
 - LEFT JOIN → all rows from left + matches.
 - RIGHT JOIN → all rows from right + matches.
- **Aggregate Functions**
 - COUNT(), SUM(), AVG(), MAX(), MIN().
- **GROUP BY / HAVING**
 - Groups rows by column values.
 - HAVING filters grouped results.
- **Subqueries**
 - Nested queries inside another query.
- **Operators**
 - LIKE, IN, BETWEEN, IS NULL, ANY, ALL, UNION.



Coding Round Questions

- Create table → Write SQL to create a students table with id, name, and course.
- Alter table → Add a column email to the students table.
- Insert data → Insert 3 students into the students table.
- Update data → Update the course of student with id=2 to "Java".
- Delete data → Delete student with id=1.
- Select query → Fetch all students enrolled in "Java".
- Transaction control → Demonstrate BEGIN, ROLLBACK, and COMMIT.
- Grant privileges → Grant SELECT and INSERT privileges on students to a user.
- Group by → Find number of employees per department.
- Aggregate functions → Find average salary per department.
- Joins → Write a query to join contractors and projects tables.
- Subquery → Find employees earning more than the average salary.
- Special operators → Find employees whose names start with "A".



Machine Coding Round Challenge

Problem: Company Database Management

Requirements:

1. Create database company_db.
2. Create tables:
 - departments(department_name, location)
 - employees(id, name, department, salary, hire_date)
 - projects(project_id, project_name, client_email)
 - contractors(contractors_id, name, assigned_project_id) with foreign key.
3. Insert sample data for each table.
4. Write queries to:
 - Fetch all employees.
 - Count employees per department.
 - Find average salary per department.
 - Find employees earning more than average salary.
 - Join contractors with projects.
 - Find employees whose names start with "A".
 - Find employees with salary between 80,000 and 100,000.
 - Show contractors with no assigned project (IS NULL).